

WebMind

MVP Software Requirements Specification (SRS)

Version: 1.0 **Date:** September 2, 2026

1. Introduction

1.1 Purpose

This SRS defines the functional and non-functional requirements for the WebMind MVP browser extension and its supporting backend. It is intended to guide architecture, implementation, testing, and acceptance.

1.2 Scope

The system consists of a browser extension, backend API, relational database, and AI service integration. The extension operates in the context of the active webpage and provides persistent AI Q&A, notes, highlights, and history.

2. System Actors

Actor	Responsibilities
User	Authenticates, browses pages, asks questions, creates notes, saves highlights, and searches history.
AI Service	Generates responses from supplied page/conversation context.
Browser	Provides active-tab/page context and hosts the extension.
Backend	Authenticates users, persists data, prepares AI context, and exposes APIs.

3. System Architecture

Conceptual flow: Browser Page → Extension Content/Side Panel → Backend API → PostgreSQL / optional pgvector → AI Service → Backend → Extension UI.

The extension should minimize direct exposure of private backend credentials. AI provider credentials should remain server-side.

4. Functional Requirements

FR-01 Authentication

- The system shall allow a user to sign up or sign in.
- The backend shall issue a secure authenticated session/token.
- The system shall prevent one user from reading another user's private data.
- The extension shall handle expired sessions and require re-authentication when necessary.

FR-02 Active Page Identification

- The extension shall obtain the active tab URL and page title.
- The extension shall identify the page's domain.
- The extension shall normalize URLs where possible.

- The system shall create or reuse a page record for the identified page.
- The extension shall extract readable page content for supported pages.

FR-03 Page Content Processing

- The extension/backend pipeline shall convert supported page content into usable text.
- Large content shall be truncated, chunked, or otherwise processed within model/context limits.
- The system shall retain enough metadata to identify the source page.
- The system shall gracefully handle pages whose content cannot be extracted.

FR-04 AI Questions

- The user shall be able to submit a question from the sidebar.
- The backend shall receive the page identity and question.
- The AI request shall include relevant current-page context.
- The system may include recent conversation messages as conversational context.
- The AI answer shall be returned to the extension and displayed.
- The question and answer shall be persisted.

FR-05 Conversations

- A page may have multiple conversations.
- A conversation shall belong to exactly one user.
- A conversation shall be associated with a page.
- Messages shall preserve role, content, timestamps, and ordering.
- Users shall be able to reopen previous conversations.

FR-06 Notes

- The user shall be able to create a note for the active page.
- The user shall be able to edit and delete their notes.
- Notes shall contain user ID, page ID, content, created time, and updated time.
- The system should optionally store the selected text/source position when a note originated from a selection.

FR-07 Highlights

- The extension shall detect user text selection where permissions allow.
- The user shall be able to save the selection as a highlight.
- A highlight shall store the selected text and associated page.
- The system should store enough location metadata to attempt to reconnect the highlight to the page.

FR-08 Page Workspace

- When a page is opened, the extension shall load its associated workspace.
- The workspace shall show available conversations, notes, and highlights.
- The workspace shall show basic activity metadata such as last activity.

FR-09 Global History/Search

- The user shall be able to search their saved notes and conversation content.
- Search results shall include source page information.
- Selecting a result shall open or navigate toward the associated page/workspace.
- MVP may use PostgreSQL full-text search; vector/semantic search is optional but recommended for later expansion.

FR-10 Error Handling

- The extension shall show useful errors when the page cannot be read.
- AI timeouts and provider errors shall not silently discard the user's question.
- Network failures shall provide retry behavior where appropriate.
- Unauthorized requests shall return an appropriate authentication error.

5. Data Requirements

Recommended relational schema:

Table	Key fields
users	id, email, password/auth reference, created_at, updated_at
pages	id, user_id, canonical_url, url, domain, title, content_hash, created_at, updated_at
conversations	id, user_id, page_id, title, created_at, updated_at
messages	id, conversation_id, role, content, created_at, sequence/order
notes	id, user_id, page_id, content, source_text, created_at, updated_at
highlights	id, user_id, page_id, selected_text, selector/location_metadata, created_at
embeddings (optional)	id, user_id, source_type, source_id, embedding, created_at

6. API Requirements

Method / endpoint	Purpose
POST /auth/signup	Create account.
POST /auth/login	Authenticate user.
POST /pages/resolve	Create/reuse a page identity from current-page metadata.
GET /pages/:id/workspace	Load conversations, notes, highlights, and page metadata.
POST /pages/:id/conversations	Create a conversation.
POST /conversations/:id/messages	Submit a question and generate/store an AI answer.
GET /pages/:id/conversations	List page conversations.
POST /pages/:id/notes	Create a note.
PATCH /notes/:id	Edit a note.
DELETE /notes/:id	Delete a note.
POST /pages/:id/highlights	Save a highlight.
DELETE /highlights/:id	Delete a highlight.

Method / endpoint	Purpose
GET /search?q=...	Search user's saved knowledge.

7. Browser Extension Requirements

- Use Manifest V3.
- Provide a side-panel/sidebar interface.
- Request only permissions required for page context and user actions.
- Content scripts should extract page-readable content without unnecessarily collecting unrelated data.
- The extension shall clearly indicate when page content is being sent for AI processing.
- The UI shall remain usable on common desktop Chromium browsers.

8. AI Context Requirements

- The backend shall construct prompts from trusted system instructions, page content, and user conversation.
- Untrusted webpage content shall be treated as data, not as system instructions.
- The AI pipeline should limit context to relevant content when pages are large.
- The system should preserve source metadata for future citation/traceability.
- The backend should apply request limits and token/cost controls.

9. Non-Functional Requirements

9.1 Security

- All client-server traffic shall use HTTPS in production.
- Passwords, if managed by the application, shall be securely hashed; a managed auth provider may be used instead.
- Users shall only access records they own.
- AI provider secrets shall never be shipped in the extension.
- Sensitive logs shall not expose user page content unnecessarily.

9.2 Performance

- Sidebar should open quickly without blocking the webpage.
- Previously stored page workspace should load independently from slow AI generation.
- AI responses should stream or provide visible progress when supported.
- Large page processing should be asynchronous where practical.

9.3 Reliability

- Persist user-created notes/highlights before performing non-essential AI operations.
- Backend should use database transactions for related writes.
- AI provider failures should not corrupt conversation history.

9.4 Privacy

- Only collect page information required for the product's stated functions.

- Provide a clear privacy policy and user-facing explanation of AI/page-content processing.
- Provide deletion mechanisms for user data in a later MVP-hardening milestone if not present at launch.

10. Security & Privacy Considerations

WebMind processes potentially sensitive browsing content. The architecture should therefore use least-privilege browser permissions, server-side AI credentials, authenticated API access, strict row-level authorization, encrypted transport, and careful logging. Page content may contain malicious prompt-injection instructions; those instructions must never override system or application policies.

11. Testing Requirements

- Unit tests for URL normalization, page identity, note/highlight CRUD, and API authorization.
- Integration tests for page → question → AI response → persistence.
- Extension tests for active-tab detection and sidebar behavior.
- Security tests for cross-user data access.
- Failure tests for unavailable pages, network errors, AI timeouts, and expired authentication.
- Manual tests across representative article, documentation, search, and dynamic pages.

12. Acceptance Criteria

- A new user can authenticate and open the extension on a supported page.
- The current page is correctly identified and shown in the sidebar.
- A user can ask a page question and receive an answer grounded in the page.
- Closing and reopening the extension does not lose the conversation.
- A note created on Page A does not appear on unrelated Page B.
- A saved highlight is associated with the correct page.
- Returning to Page A restores its notes, highlights, and conversation history.
- Global search returns the user's saved content and source pages.
- A user cannot access another user's pages, conversations, notes, or highlights.

13. Recommended MVP Delivery Phases

Phase	Deliverables
1. Foundation	Extension shell, authentication, backend, PostgreSQL schema, page identification.
2. AI Q&A	Page extraction, context pipeline, AI endpoint, streaming/response UI, conversation persistence.
3. Notes & Highlights	Selection actions, note CRUD, highlight persistence, page workspace.
4. History/Search	Page history UI, global search, workspace restoration.
5. Hardening	Security, permissions, rate limits, error states, tests, privacy controls, browser compatibility.

14. Future Architecture Direction

The MVP database should be designed so that semantic embeddings can be added without restructuring the core entities. A future retrieval layer can index page content, notes, highlights, and conversations, enabling cross-page

questions such as: “What have I learned about this topic?” and “Compare the three pages I read.”